Syncner Application Setup Guide

This guide will help you set up the Syncner learning management system from scratch using the provided zip file.

Table of Contents

1. Prerequisites
2. Environment Setup
3. Database Configuration
4. Backend Setup
5. Frontend Setup
6. Application Configuration
7. Running the Application
8. Troubleshooting
9. Production Deployment

Prerequisites

Before setting up the application, ensure you have the following installed:

Required Software

- **Node.js** (v18.0.0 or higher)
- **npm** (v8.0.0 or higher) or **yarn** (v1.22.0 or higher)
- **MongoDB** (v5.0 or higher)
- **Git** (for version control)

Optional but Recommended

- **MongoDB Compass** (for database management)
- **Postman** (for API testing)
- **VS Code** (for development)

Environment Setup

1. Extract the Zip File

```
# Extract the zip file to your desired location
unzip syncner-application.zip
cd syncner-application
```

2. Create Environment Files

Backend Environment (.env)

Create a `.env` file in the `backend/` directory:

```
# Database Configuration
MONGODB_URI=mongodb://localhost:27017/syncner

# JWT Configuration
JWT_SECRET=your-super-secret-jwt-key-here-make-it-long-and-random

# Server Configuration
PORT=5000
NODE_ENV=development

# CORS Configuration
FRONTEND_URL=http://localhost:3000

# Email Configuration (Optional - for password reset)
EMAIL_HOST=smtp.gmail.com
EMAIL_PORT=587
EMAIL_USER=your-email@gmail.com
EMAIL_PASS=your-app-password

# File Upload Configuration
MAX_FILE_SIZE=5242880
UPLOAD_PATH=./uploads
```

Frontend Environment (.env)

Create a `.env` file in the root directory:

```
VITE_API_BASE_URL=http://localhost:5000
VITE_APP_NAME=Syncner
VITE_APP_VERSION=1.0.0
```

Database Configuration

1. Install and Start MongoDB

On Windows:

1. Download MongoDB from mongodb.com
2. Install MongoDB Community Server
3. Start MongoDB service:

```
net start MongoDB
```

On macOS:

```
# Using Homebrew
brew tap mongodb/brew
brew install mongodb-community
brew services start mongodb/brew/mongodb-community
```

On Linux (Ubuntu/Debian):

```
# Import MongoDB public key
wget -qO - https://www.mongodb.org/static/pgp/server-6.0.asc | sudo apt-
key add -

# Create list file
echo "deb [ arch=amd64,arm64 ] https://repo.mongodb.org/apt/ubuntu
focal/mongodb-org/6.0 multiverse" | sudo tee
/etc/apt/sources.list.d/mongodb-org-6.0.list

# Update package database
sudo apt-get update

# Install MongoDB
sudo apt-get install -y mongodb-org

# Start MongoDB
sudo systemctl start mongod
sudo systemctl enable mongod
```

2. Verify MongoDB Installation

```
# Check if MongoDB is running
mongosh --version

# Connect to MongoDB
mongosh
```

Backend Setup

1. Navigate to Backend Directory

```
cd backend
```

2. Install Dependencies

```

```

```
npm install
# or
yarn install
```

3. Database Migration (Optional)

If you want to run the database migrations:

```
# Fix user progress level orders
node utils/fixUserProgressLevelOrders.js

# Fix career tracks
node migrations/fixCareerTracks.js

# Fix progress level orders
node migrations/fixProgressLevelOrders.js
```

4. Seed Database (Optional)

To populate the database with sample data:

```
node seedData.js
```

5. Start Backend Server

```
# Development mode
npm run dev
# or
yarn dev

# Production mode
npm start
# or
yarn start
```

The backend server will start on `http://localhost:5000`

Frontend Setup

1. Navigate to Root Directory

```
cd ..
# (from backend directory, go back to root)
```

2. Install Dependencies

```
npm install
# or
yarn install
```

3. Start Frontend Development Server

```
# Development mode
npm run dev
# or
yarn dev

# Build for production
npm run build
# or
yarn build
```

The frontend will start on `http://localhost:3000`

Application Configuration

1. Default Admin Account

After seeding the database, you can log in with:

- **Email**: admin@syncner.com
- **Password**: admin123

2. User Roles

The application supports three user roles:

- **Admin**: Full system access
- **Staff**: Student management and analytics
- **Student**: Learning interface

3. Career Tracks

The system supports multiple career tracks:

- Full-Stack Development
- Data Science
- DevOps
- UI/UX Design

Running the Application

Development Mode

1. Start MongoDB
2. Start Backend: `cd backend && npm run dev`
3. Start Frontend: `npm run dev`
4. Open browser to `http://localhost:3000`

Production Mode

1. Build Frontend: `npm run build`
2. Start Backend: `cd backend && npm start`
3. Serve frontend build files using a web server (nginx, Apache, etc.)

Troubleshooting

Common Issues

1. MongoDB Connection Error

```
Error: connect ECONNREFUSED 127.0.0.1:27017
```

Solution: Ensure MongoDB is running and the connection string is correct.

2. Port Already in Use

```
Error: listen EADDRINUSE :::5000
```

Solution: Change the PORT in `.env` file or kill the process using the port.

3. JWT Secret Error

```
Error: secretOrPrivateKey must have a value
```

Solution: Ensure JWT_SECRET is set in the backend `.env` file.

4. CORS Error

```
Access to fetch at 'http://localhost:5000' from origin
'http://localhost:3000' has been blocked by CORS policy
```

Solution: Check CORS configuration in backend and ensure FRONTEND_URL is set correctly.

5. File Upload Issues

```
Error: ENOENT: no such file or directory, open './uploads/...'
```

Solution: Create the uploads directory: `mkdir backend/uploads`

Debug Mode

To enable debug logging, set `NODE_ENV=development` in your backend `.env` file.

Database Issues

If you encounter database-related issues:

1. Check MongoDB connection
2. Verify database name in MONGODB_URI
3. Run database migrations
4. Check for duplicate data

Production Deployment

1. Environment Variables

Update your production `.env` files with:

- Production MongoDB URI
- Strong JWT secret
- Production CORS settings
- Email configuration for notifications

2. Database Security

- Enable MongoDB authentication
- Use strong passwords
- Configure firewall rules
- Enable SSL/TLS

3. Server Configuration

- Use PM2 for process management
- Configure reverse proxy (nginx)
- Enable HTTPS
- Set up monitoring and logging

4. Frontend Deployment

- Build the frontend: `npm run build`
- Serve static files through a web server
- Configure CDN for assets

- Enable gzip compression

API Documentation

Base URL

- Development: `http://localhost:5000`
- Production: `https://your-domain.com`

Authentication

All protected routes require a JWT token in the Authorization header:

```
Authorization: Bearer <your-jwt-token>
```

Key Endpoints

- `POST /api/auth/login` - User login
- `POST /api/auth/register` - User registration
- `GET /api/students` - Get all students (Staff/Admin)
- `GET /api/levels` - Get learning levels
- `POST /api/progress/mark-completed` - Mark level as completed

File Structure

```
syncner-application/
├── backend/
│   ├── controllers/     # API route handlers
│   ├── models/         # Database models
│   ├── routes/         # API routes
│   ├── middleware/     # Custom middleware
│   ├── services/       # Business logic
│   ├── utils/          # Utility functions
│   ├── migrations/     # Database migrations
│   └── uploads/        # File uploads
├── src/
│   ├── components/     # React components
│   ├── pages/         # Page components
│   ├── hooks/         # Custom React hooks
│   ├── redux/         # State management
│   ├── services/      # API services
│   └── utils/         # Utility functions
├── docs/              # Documentation
└── public/            # Static assets
```

Support

If you encounter any issues during setup:

1. Check the troubleshooting section above
2. Verify all prerequisites are installed
3. Ensure all environment variables are set correctly
4. Check the application logs for detailed error messages
5. Refer to the API documentation for endpoint details

Additional Resources

- MongoDB Documentation
- Node.js Documentation
- React Documentation
- Express.js Documentation

Note: This setup guide assumes you're starting with a fresh installation. If you're migrating from an existing setup, please refer to the migration documentation in the `docs/` directory.